

Министерство науки и высшего образования Российской Федерации
Федеральное государственное автономное образовательное учреждение
высшего образования
«МОСКОВСКИЙ ПОЛИТЕХНИЧЕСКИЙ УНИВЕРСИТЕТ»
(МОСКОВСКИЙ ПОЛИТЕХ)
КАФЕДРА СМАРТ-ТЕХНОЛОГИИ

Отчет по выполнению лабораторной работы №6
«Параметрическое построение сопла Лаваля в Salome для анализа
течения газа с использованием инструментария вычислительной
гидродинамики OpenFOAM»
по дисциплине «Системы инженерного анализа»
по направлению 09.03.01 Информатика и вычислительная техника
Образовательная программа (профиль)
«Интеграция и программирование в САПР»

Преподаватель: / _____ / Лаврененко Илья Станиславович /
подпись *ФИО*

Студент: / _____ / Старков Матвей Антонович, 231-324 /
подпись *ФИО, группа*

Москва, 2026

ОПИСАНИЕ ВЫПОЛНЕННОЙ РАБОТЫ

Открываем *SALOME* и переходим в модуль *Shaper*, создаем новое решение и добавляем переменные для построения профиля конического сопла Лавали.

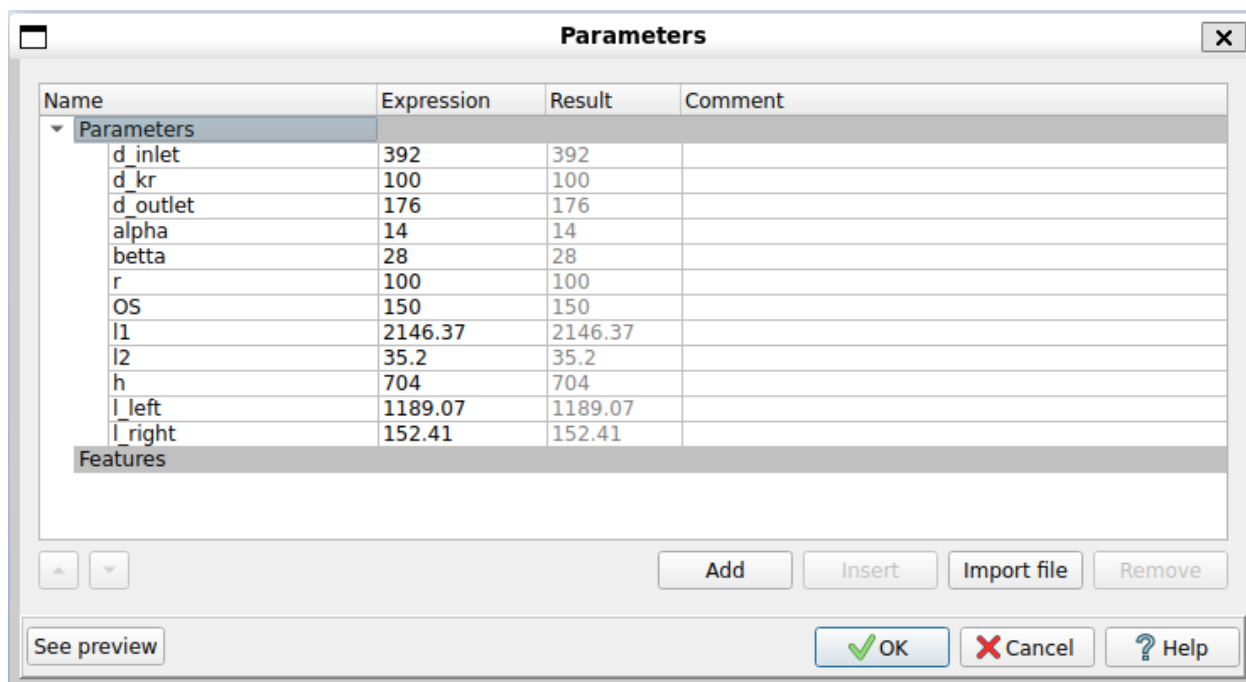


Рисунок 1 – Создание переменных в SALOME

Создаем эскиз и строим геометрию на введенных ранее параметрах.

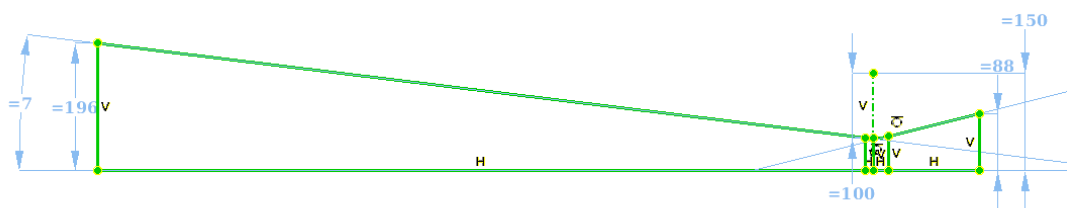


Рисунок 2 – Эскиз конического сопла Лавали



Рисунок 3 – Завершенный эскиз конического сопла Лавали с расчетной областью

Полученный эскиз выдавливаем вращением для получения 3d модели.

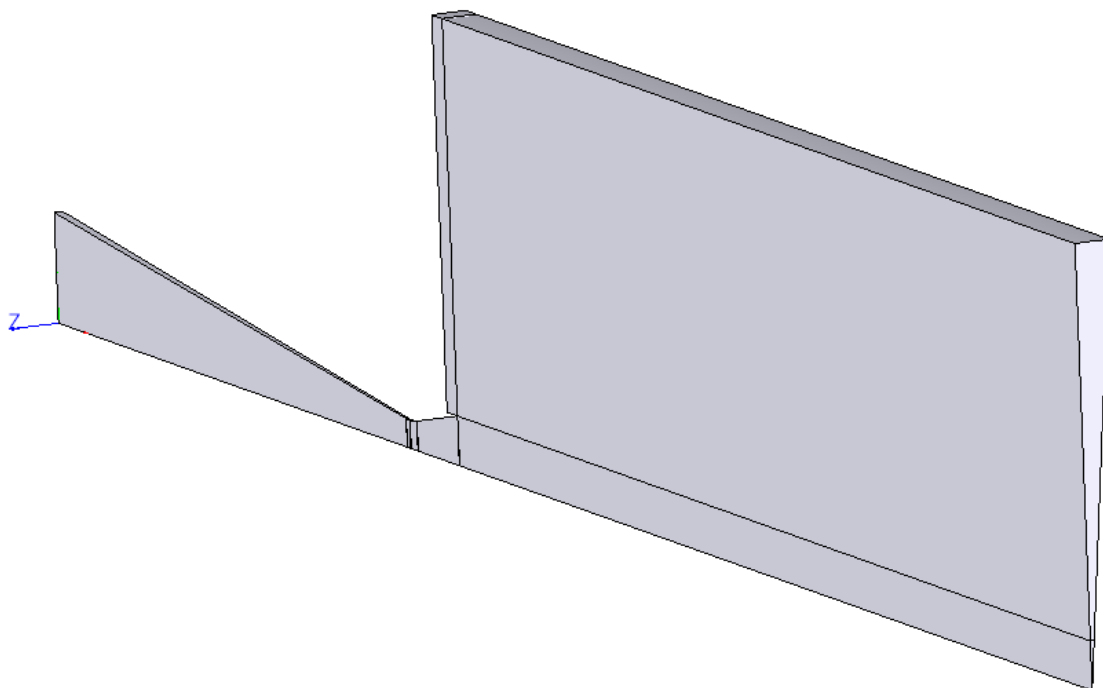


Рисунок 4 – Результат выдавливания эскиза вращением

Создадим группы, это подготовительный этап перед созданием сетки.

Так же сохраним скрипт на *python*. На этом создание модели закончено.



Рисунок 5 – Структура модели

Настраиваем сетку и экспортируем в формате *UNV*, так же сохраняем скрипт для дальнейшей модификации.

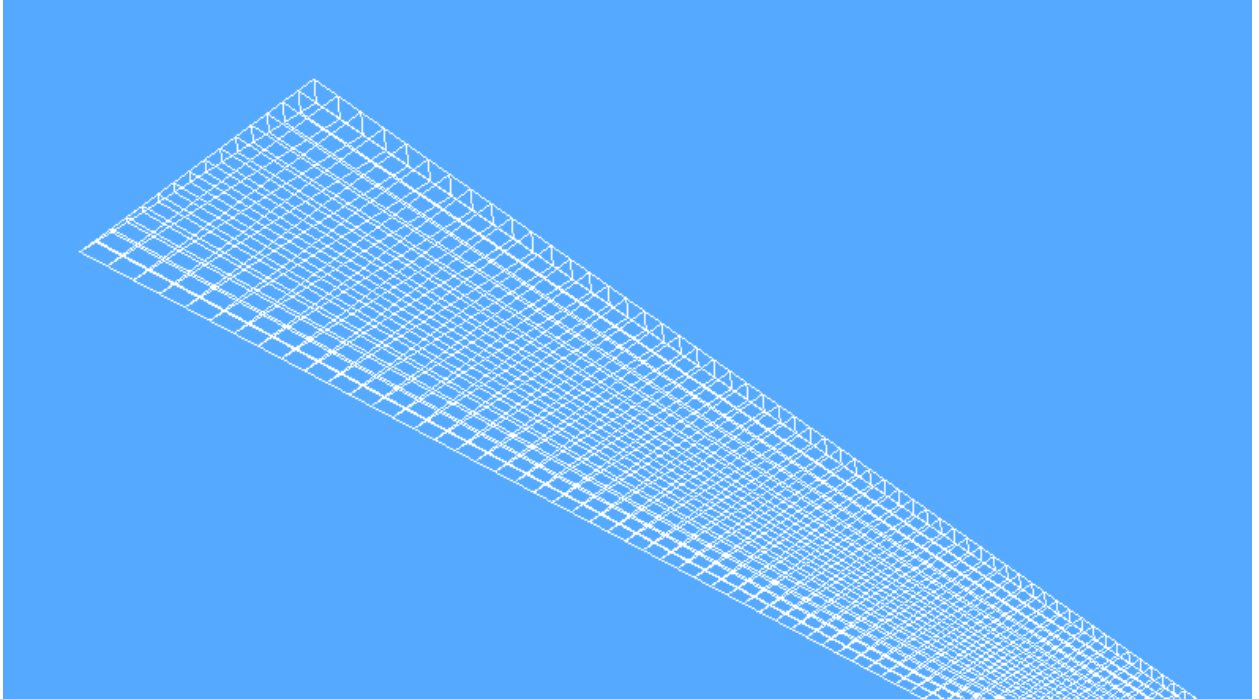


Рисунок 6 – Часть созданной сетки

В файл скрипта добавляем входные параметры.

Листинг 1 – Входные параметры в script.py

```
# входные параметры  
p_input = 200000  
T_input = 1800  
P_output = 9000  
G = 1.5  
alpha_ = 14  
beta_ = 28  
R = 287  
k = 1.4
```

Далее последовательно выполним расчет входного сечения сопла.

Листинг 2 – Расчет входного сечения

```
# расчет входного сечения
```

```

beta_vh = 0.999
# давление
p_vh = beta_vh * P_input
# температура
T_vh = T_input * beta_vh ** ((k - 1) / k)
# удельный объем
mu_vh = R * T_vh / p_vh
# плотность
ro_vh = 1 / mu_vh
# скорость потока
c_vh = math.sqrt(2 * k / (k - 1) * R * (T_input - T_vh))
# местная скорость звука
a_vh = math.sqrt(k * R * T_vh)
# площадь входного поперечного сечения
F_vh = G / ro_vh / c_vh
# диаметр входного сечения сопла
d_vh = round(math.sqrt(4 / 3.14 * F_vh) * 1e3)

```

Аналогичным образом выполним расчет для критического и выходного сечения.

Листинг 3 – Расчет критического сечения

```

# расчет критического сечения
beta_kr = (2 / (k + 1)) ** (k / (k - 1))
# давление
p_kr = beta_kr * P_input
# температура
T_kr = T_input * beta_kr ** ((k - 1) / k)
# удельный объем
mu_kr = R * T_kr / p_kr
# плотность

```

```

ro_kr = 1 / mu_kr
# СКОРОСТЬ ПОТОКА
c_kr = math.sqrt(2 * k / (k - 1) * R * (T_input - T_kr))
# местная скорость звука
a_kr = math.sqrt(k * R * T_kr)
# площадь критического поперечного сечения
F_kr = G / ro_kr / c_kr
# диаметр критического сечения сопла
d_kr = round(math.sqrt(4 / 3.14 * F_kr) * 1e3)

```

Листинг 5 – Расчет выходного сечения

```

# расчет выходного сечения
beta_out = P_output / P_input
# давление
p_out = beta_out * P_input
# температура
T_out = T_input * beta_out ** ((k - 1) / k)
# удельный объем
mu_out = R * T_out / p_out
# плотность
ro_out = 1 / mu_out
# скорость потока
c_out = math.sqrt(2 * k / (k - 1) * R * (T_input - T_out))
# местная скорость звука
a_out = math.sqrt(k * R * T_out)
# площадь выходного поперечного сечения
F_out = G / ro_out / c_out
# диаметр выходного сечения сопла
d_out = round(math.sqrt(4 / 3.14 * F_out) * 1e3)

```

Следующим этапом выполним расчет длины сопла дозвуковой и сверхзвуковой частей. Не забываем переводить в радианы.

Листинг 6 – Вычисление длин сопла до критического сечения и после

```
# расчетная длина сопла до критического сечения и после
l_left = (d_vh - d_kr) / (2 * math.tan(alpha_ / 2 * math.pi / 180))
l_right = (d_out - d_kr) / (2 * math.tan(betta_ / 2 * math.pi / 180))
```

Далее введем локальные переменные, которые позже будем передавать в *notebook*.

Листинг 7 – Создание локальных переменных

```
d_inlet_ = d_vh
d_kr_ = d_kr
d_outlet_ = d_out

r_ = d_kr
OS_ = r_ + d_kr_ / 2
l1_ = (l_left + l_right) * 1.6
l2_ = d_outlet_ / 5
h_ = d_outlet_ * 4
```

Далее вычислим длины дуг около критического сечения для того, чтобы подстраивать количество ячеек при изменении входных параметров.

Листинг 8 – Вычисление длин дуг

```
arc_left = math.pi * alpha_ * r_ / 180
arc_right = math.pi * betta_ * r_ / 180
```

Следующим шагом вычислим коэффициенты по каждому направлению для параметрического перестроения сетки.

Листинг 9 – Вычисление коэффициентов

```
# atmosphere height above nozzle
k_height = 177 * 6 / 885
```

```
# atmosphere width
k_width = (1189.09 + 152.41 + 2146.4) / 2146.4
# left top atmosphere
k_left_top = (2146.4 + 30.482) / 30.482
# nozzle width
k_nozzle = (1189.09 + 152.41) / 1189.09
# arc left
k_arc_left = (14 * 3.14 * 100) / 180 / 628
# arc right
k_arc_right = (28 * 3.14 * 100) / 180 / 628
```

Далее вычислим количество ячеек для каждой группы.

Листинг 10 – Создание переменных, отвечающих за количество ячеек, используемых в гипотезах

```
NbSg1 = 15
NbSg2 = 1
NbSg3 = round(l_left * k_nozzle / 13.4)

NbSg4 = round(arc_left * k_arc_left * 2)
NbSg5 = round(arc_right * k_arc_right)

NbSg6 = round(l_right * k_nozzle / 5.7)
NbSg7 = round(l1_ * k_width / 7)
NbSg8 = round(l2_ * k_left_top / 360)
NbSg9 = round(h_ * k_height / 15)
```

Вносим изменения в значения, которые будут использоваться при построении, задаем значения локально вычисленных переменных.

Листинг 11 – Изменение параметров на переменные

```
Part_2 = model.addPart(partSet)
```

```

Part_2_doc = Part_2.document()
model.addParameter(Part_2_doc, "d_inlet", "d_inlet_")
model.addParameter(Part_2_doc, "d_kr", "d_kr_")
model.addParameter(Part_2_doc, "d_outlet", "d_outlet_")
model.addParameter(Part_2_doc, "alpha", "alpha_")
model.addParameter(Part_2_doc, "betta", "betta_")
model.addParameter(Part_2_doc, "r", "r_")
model.addParameter(Part_2_doc, "OS", "OS_")
model.addParameter(Part_2_doc, "l1", "l1_")
model.addParameter(Part_2_doc, "l2", "l2_")
model.addParameter(Part_2_doc, "h", "h_")
model.addParameter(Part_2_doc, "l_left", "l_left")
model.addParameter(Part_2_doc, "l_right", "l_right")

```

Переходим к модулю SMESH. Находим строки, отвечающие за количество ячеек, и последовательно меняем значения на переменные, которые были обозначены ранее.

Листинг 12 – Изменение констант на переменные

```

Number_of_Segments_1 = Regular_1D_1.NumberOfSegments(NbSg2, None, [])
Number_of_Segments_100 = Regular_1D_3.NumberOfSegments(NbSg3, None,
[])
Number_of_Segments_2 = Regular_1D_4.NumberOfSegments(NbSg4, None, [])
Number_of_Segments_4 = Regular_1D_5.NumberOfSegments(NbSg5, None, [])
Number_of_Segments_30 = Regular_1D_6.NumberOfSegments(NbSg6, None,
[])
Number_of_Segments_500 = Regular_1D_7.NumberOfSegments(NbSg7, None,
[])
Number_of_Segments_7 = Regular_1D_8.NumberOfSegments(NbSg8, None, [])
Number_of_Segments_56 = Regular_1D_9.NumberOfSegments(NbSg9, None, [])

```

Добавим экспорт сетки.

Листинг 13 – Экспорт сетки

```
try:
    Mesh_1.ExportUNV(r'/home/matvey/OpenFOAM/matvey-
13/run/nozzle_1/geometry/Mesh.unv')
    pass
except:
    print('ExportUNV() failed!')
```

При загрузке скрипта в *SALOME* сетка должна автоматически построиться с нужными параметрами.

Копируем учебный пример, который будем использовать.

Листинг 14 – Копирование файлов

```
run
cp -rf $FOAM_TUTORIALS/shockFluid/shockTube nozzle_1
```

В каталоге *constant* настраиваем файл *physicalProperties*, в файле *momentumTransport* просто указано, что поток является ламинарным.

Листинг 15 – Файл *physicalProperties*

```
thermoType
{
    type        hePsiThermo;
    mixture     pureMixture;
    transport    const;
    thermo       hConst;
    equationOfState perfectGas;
    specie       specie;
    energy       sensibleInternalEnergy;
}
```

```

mixture
{
  specie
  {
    molWeight    28.9;
  }
  thermodynamics
  {
    Cp           1005;
    hf           0;
  }
  transport
  {
    mu           0;
    Pr           0.7;
  }
}

```

Теперь перейдем в каталог *0* для настройки начальных и граничных условий. Откроем файл скорости и создадим там граничные условия. Передней и задней частям укажем тип *wedge*, так как форма геометрии напоминает клин с малым углом, стенка обозначена адиабатическая без прилипания, входной и выходной граням установим нулевой градиент. Изначально скорость будет равна нулю. Сохраним изменения в файле и переименуем его просто в *U*.

Листинг 16 – Файл *U*

```

dimensions    [0 1 -1 0 0 0 0];

internalField  uniform (0 0 0);

```

```
boundaryField
{
    inlet
    {
        type        zeroGradient;
    }

    outletRight
    {
        type zeroGradient;
    }
    outletTop
    {
        type zeroGradient;
    }
    outletLeft
    {
        type    zeroGradient;
    }

    wall
    {
        type        noSlip;
    }

    front
    {
        type wedge;
    }
}
```

```
back
{
    type wedge;
}
}
```

Теперь настроим файл температуры. По условия на входе воздух имеет температуру 1800 К, установим это значение в качестве величины внутреннего 35 поля, фиксированного значения грани на входе, а также грани выхода с левой стороны. Остальные выходы и стена по умолчанию будут иметь нулевой градиент, передняя и задняя стенки, как и в файле скорости, задаются типом wedge.

Листинг 17 – Файл T

```
dimensions    [0 0 0 1 0 0 0];

internalField  uniform 1800;

boundaryField
{
    inlet
    {
        type      fixedValue;
        value      uniform 1800;
    }

    outletRight
    {
        type zeroGradient;
    }

    outletTop
```

```

{
    type zeroGradient;
}
outletLeft
{
    type      fixedValue;
    value     uniform 1800;
}

wall
{
    type      zeroGradient;
}

front
{
    type      wedge;
}
back
{
    type      wedge;
}
}

```

Файл *p.orig* изменяем по аналогии с файлом температуры, но сохраняем название, так как начальное распределение будет модифицировать записи в нем.

Теперь перейдем в каталог *system* и настроим системные файлы. Удалим два файла: *blockMeshDict* и *sample*. Первый нам не нужен, так как сетку мы экспортируем из *Salome*, а не строим с помощью утилиты *blockMesh*. Второй

файл не нужен, так как постобработку пользователь будет выполнять сам.
Настраиваем файлы *fvSchemes* и *fvSolution*.

Листинг 18 – Файл fvSchemes

```
ddtSchemes
{
    default      Euler;
}

gradSchemes
{
    default      Gauss linear;
}

divSchemes
{
    default      none;

    div(phi,U)      Gauss upwind;
    div(phiid,p)     Gauss limitedLinear 1;
    div(phi,e)       Gauss limitedLinear 1;
    div(phi,K)       Gauss limitedLinear 1;
    div(phi,(p|rho)) Gauss limitedLinear 1;

    div((((rho*nuEff)*dev2(T(grad(U)))))) Gauss linear;
}

laplacianSchemes
{
    default      Gauss linear orthogonal;
```

```

}

interpolationSchemes
{
    default    linear;
}

snGradSchemes
{
    default    orthogonal;
}

```

Листинг 19 – Файл fvSolution

```

ddtSchemes
{
    default    Euler;
}

gradSchemes
{
    default    Gauss linear;
}

divSchemes
{
    default    none;

    div(phi,U)    Gauss upwind;
    div(phi,d,p)   Gauss limitedLinear 1;
    div(phi,e)     Gauss limitedLinear 1;
}

```

```

div(phi,K)      Gauss limitedLinear 1;
div(phi,(p|rho)) Gauss limitedLinear 1;

div((((rho*nuEff)*dev2(T(grad(U)))))) Gauss linear;
}

laplacianSchemes
{
    default      Gauss linear orthogonal;
}

interpolationSchemes
{
    default      linear;
}

snGradSchemes
{
    default      orthogonal;
}

```

Теперь откроем файл *controlDict*, настроим время решения. В качестве параметра *startFrom* установим *latestTime*, чтобы расчет начинался с последнего времени, для которого задача была решена. Параметр *endTime* установим 0.05, *deltaT* немного уменьшим до 1e-06. Записывать результаты будем чаще, поэтому параметр *writeInterval* необходимо задать значение 1e04. Параметр *writeFormat* изменяем на *binary*, чтобы избежать ошибок при запуске расчета, и увеличиваем значение *writePrecision* до 10.

Изменим файл *setFieldsDict* для настройки начального распределения. Для этого нужно импортировать сетку и открыть ее в *paraView*, чтобы

получить координаты точек, которые образуют дозвуковую часть сопла.

Листинг 20 – Файл `setFieldsDict`

```
defaultFieldValues
(
    volScalarFieldValue p 9000
);

regions
(
    boxToCell
    {
        box (0 0 -1) (1.2 0.2 1);
        fieldValues
        (
            volScalarFieldValue p 2e5
        );
    }
);
```

Убедимся, что на момент времени 0, давление распределено до критического сечения

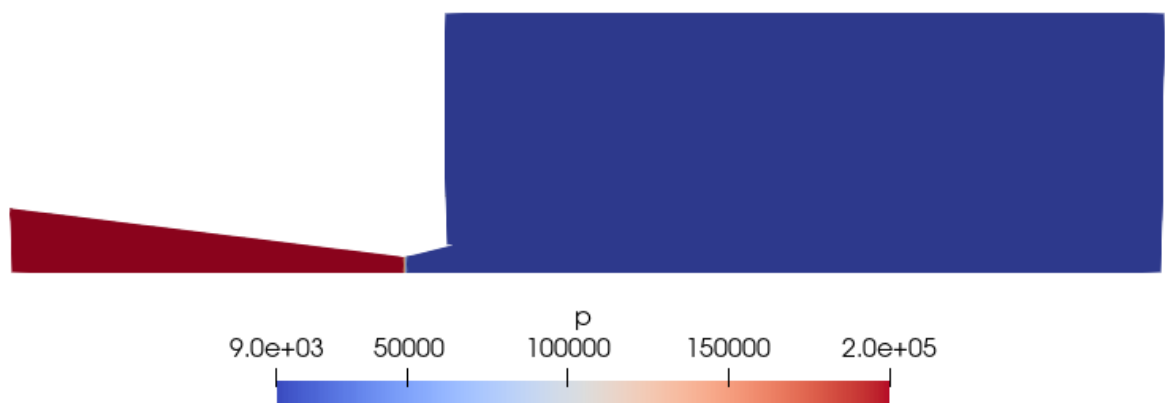


Рисунок 7 – Начальное распределение давления

Теперь в каталоге `system` создадим файл `decomposeParDict` для

обеспечения параллельного расчета.

Листинг 21 – Файл decomposeParDict

```
numberOfSubdomains 4;

method    simple;

simpleCoeffs
{
    n      (2 2 1);
}

hiererchicalCoeffs
{
    n      (1 1 1);
    order  xyz;
}

manualCoeffs
{
    dataFile  "";
}

distributed  no;

roots      ();
```

Для автоматизации решение создадим скрипт, который будет автоматически удалять файлы предыдущего решения и запускать новое.

Листинг 22 – Solve.sh

```
#!/bin/bash
```

```
rm -rf 0.*
rm -rf processor*
rm -rf constant/polyMesh
rm -rf geometry/Mesh.unv
rm 0/p
cp 0/p.orig 0/p

echo Clean done

cd ~/SALOME
./salome -t /home/matvey/OpenFOAM/Matvey-13/run/nozzle_1/geometry
/script2.py

cd /home/matvey/OpenFOAM/matvey-13/run/shockFluidTube
ideasUnvToFoam geometry/Mesh.unv
transformPoints "scale = (0.001 0.001 0.001)"
sed -i '27 s/patch/wall/' constant/polyMesh/boundary
sed -i '21 s/patch/wedge/' constant/polyMesh/boundary
sed -i '57 s/patch/wedge/' constant/polyMesh/boundary

setFields
decomposePar
mpirun -np 4 rhoPimpleFoam -parallel
reconstructPar
paraFoam
```

Все готово, остается только выполнить скрипт. Рассмотрим значения на момент времени 0.0052.

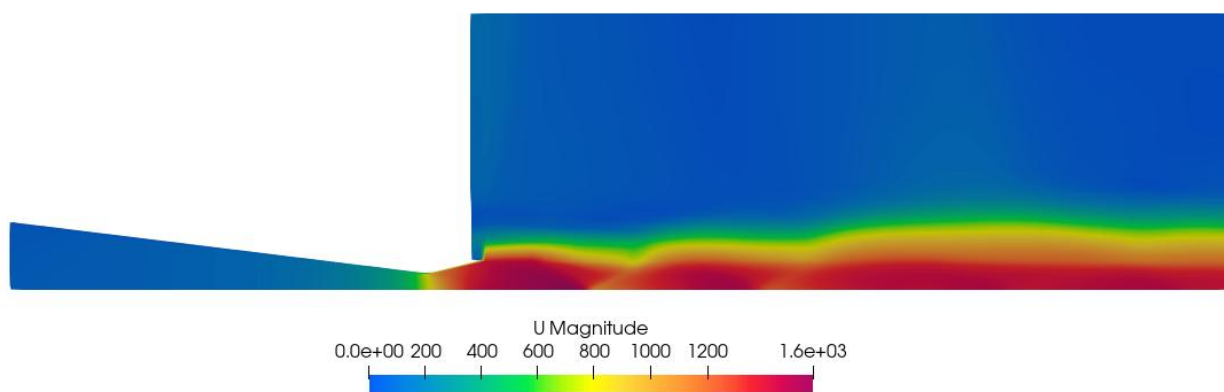


Рисунок 8 – Распределение скорости по всей длине расчетной области

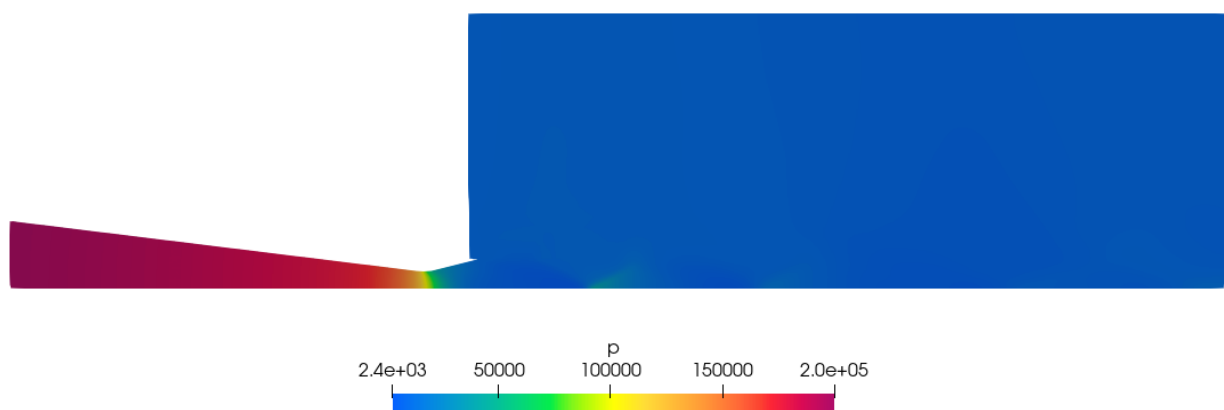


Рисунок 9 – Распределение давления

Для вычисления числа Маха используем инструмент *Calculator* в *paraView*. Добавим поле *Mach* с формулой $\text{mag}(U)/\sqrt{1.4 \cdot p/\rho}$, где 1.4 – γ для воздуха.

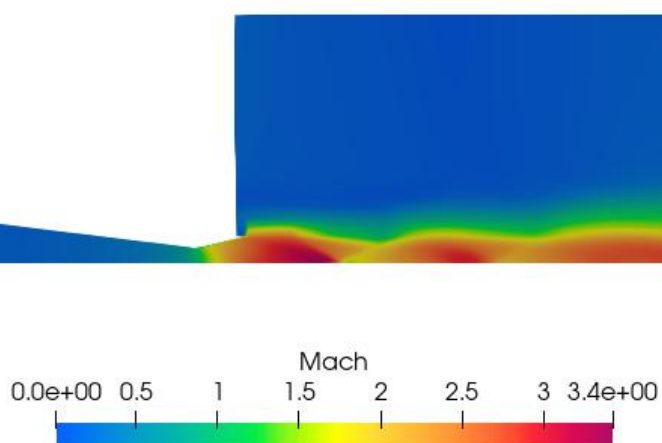


Рисунок 10 – Распределение числа Маха

ПРИЛОЖЕНИЕ А. ЛИСТИНГ КОДА SALOME

```
#!/usr/bin/env python

###
### This file is generated automatically by SALOME v9.15.0 with dump python
functionality
###

import sys
import salome
import math

salome.salome_init()
import salome_notebook
notebook = salome_notebook.NoteBook()
sys.path.insert(0, r'/home/matvey/OpenFOAM/matvey-
13/run/nozzle_1/geometry')

# входные параметры
P_input = 200000
T_input = 1800
P_output = 9000
G = 1.5
alpha_ = 14
betta_ = 28
R = 287
k = 1.4

# расчет входного сечения
```

```

beta_vh = 0.999
# давление
p_vh = beta_vh * P_input
# температура
T_vh = T_input * beta_vh ** ((k - 1) / k)
# удельный объем
mu_vh = R * T_vh / p_vh
# плотность
ro_vh = 1 / mu_vh
# скорость потока
c_vh = math.sqrt(2 * k / (k - 1) * R * (T_input - T_vh))
# местная скорость звука
a_vh = math.sqrt(k * R * T_vh)
# площадь входного поперечного сечения
F_vh = G / ro_vh / c_vh
# диаметр входного сечения сопла
d_vh = round(math.sqrt(4 / 3.14 * F_vh) * 1e3)

# расчет критического сечения
beta_kr = (2 / (k + 1)) ** (k / (k - 1))
# давление
p_kr = beta_kr * P_input
# температура
T_kr = T_input * beta_kr ** ((k - 1) / k)
# удельный объем
mu_kr = R * T_kr / p_kr
# плотность
ro_kr = 1 / mu_kr
# СКОРОСТЬ ПОТОКА

```

```

c_kr = math.sqrt(2 * k / (k - 1) * R * (T_input - T_kr))
# местная скорость звука
a_kr = math.sqrt(k * R * T_kr)
# площадь критического поперечного сечения
F_kr = G / ro_kr / c_kr
# диаметр критического сечения сопла
d_kr = round(math.sqrt(4 / 3.14 * F_kr) * 1e3)

# расчет выходного сечения
beta_out = P_output / P_input
# давление
p_out = beta_out * P_input
# температура
T_out = T_input * beta_out ** ((k - 1) / k)
# удельный объем
mu_out = R * T_out / p_out
# плотность
ro_out = 1 / mu_out
# скорость потока
c_out = math.sqrt(2 * k / (k - 1) * R * (T_input - T_out))
# местная скорость звука
a_out = math.sqrt(k * R * T_out)
# площадь выходного поперечного сечения
F_out = G / ro_out / c_out
# диаметр выходного сечения сопла
d_out = round(math.sqrt(4 / 3.14 * F_out) * 1e3)

# расчетная длина сопла до критического сечения и после
l_left = (d_vh - d_kr) / (2 * math.tan(alpha_ / 2 * math.pi / 180))

```

```

l_right = (d_out - d_kr) / (2 * math.tan(betta_ / 2 * math.pi / 180))

d_inlet_ = d_vh
d_kr_ = d_kr
d_outlet_ = d_out

r_ = d_kr
OS_ = r_ + d_kr_ / 2
l1_ = (l_left + l_right) * 1.6
l2_ = d_outlet_ / 5
h_ = d_outlet_ * 4

arc_left = math.pi * alpha_ * r_ / 180
arc_right = math.pi * betta_ * r_ / 180

# atmosphere height above nozzle
k_height = 177 * 6 / 885
# atmosphere width
k_width = (1189.09 + 152.41 + 2146.4) / 2146.4
# left top atmosphere
k_left_top = (2146.4 + 30.482) / 30.482
# nozzle width
k_nozzle = (1189.09 + 152.41) / 1189.09
# arc left
k_arc_left = (14 * 3.14 * 100) / 180 / 628
# arc right
k_arc_right = (28 * 3.14 * 100) / 180 / 628

NbSg1 = 15

```

```

NbSg2 = 1
NbSg3 = round(l_left * k_nozzle / 13.4)

NbSg4 = round(arc_left * k_arc_left * 2)
NbSg5 = round(arc_right * k_arc_right)

NbSg6 = round(l_right * k_nozzle / 5.7)
NbSg7 = round(l1_ * k_width / 7)
NbSg8 = round(l2_ * k_left_top / 360)
NbSg9 = round(h_ * k_height / 15)

###
### SHAPER component
###

from SketchAPI import *

from salome.shaper import model

model.begin()
partSet = model.moduleDocument()

### Create Part
Part_1 = model.addPart(partSet)
Part_1_doc = Part_1.document()

model.do()

```

```

#### Create Part

Part_2 = model.addPart(partSet)
Part_2_doc = Part_2.document()
model.addParameter(Part_2_doc, "d_inlet", 'd_inlet_')
model.addParameter(Part_2_doc, "d_kr", 'd_kr_')
model.addParameter(Part_2_doc, "d_outlet", 'd_outlet_')
model.addParameter(Part_2_doc, "alpha", 'alpha_')
model.addParameter(Part_2_doc, "betta", 'betta_')
model.addParameter(Part_2_doc, "r", 'r_')
model.addParameter(Part_2_doc, "OS", 'OS_')
model.addParameter(Part_2_doc, "l1", 'l1_')
model.addParameter(Part_2_doc, "l2", 'l2_')
model.addParameter(Part_2_doc, "h", 'h_')
model.addParameter(Part_2_doc, "l_left", 'l_left')
model.addParameter(Part_2_doc, "l_right", 'l_right')


#### Create Sketch

Sketch_1 = model.addSketch(Part_2_doc, model.defaultPlane("XOY"))


#### Create SketchLine

SketchLine_1 = Sketch_1.addLine(0, 196, 0, 0)
Sketch_1.setVertical(SketchLine_1.result(), True)


#### Create SketchLine

SketchLine_2 = Sketch_1.addLine(0, 0, 1183.003906157277, 0)
Sketch_1.setCoincident(SketchLine_1.endPoint(), SketchLine_2.startPoint(),
True)
Sketch_1.setHorizontal(SketchLine_2.result(), True)

```

```

#### Create SketchLine
SketchLine_3 = Sketch_1.addLine(1183.003906157277,0, 1195.190840497792,
0)
Sketch_1.setCoincident(SketchLine_2.endPoint(),    SketchLine_3.startPoint(),
True)
Sketch_1.setHorizontal(SketchLine_3.result(), True)

#### Create SketchLine
SketchLine_4 = Sketch_1.addLine(1195.190840497792,0, 1219.383030057759,
0)
Sketch_1.setCoincident(SketchLine_3.endPoint(),    SketchLine_4.startPoint(),
True)
Sketch_1.setHorizontal(SketchLine_4.result(), True)

#### Create SketchLine
SketchLine_5 = Sketch_1.addLine(1219.383030057759,0, 1359.878972062445,
0)
Sketch_1.setCoincident(SketchLine_4.endPoint(),    SketchLine_5.startPoint(),
True)
Sketch_1.setHorizontal(SketchLine_5.result(), True)

#### Create SketchLine
SketchLine_6 = Sketch_1.addLine(1359.878972062445,0, 1359.878972062445,
88)
Sketch_1.setCoincident(SketchLine_5.endPoint(),    SketchLine_6.startPoint(),
True)
Sketch_1.setVertical(SketchLine_6.result(), True)

#### Create SketchLine

```

```

SketchLine_7      =      Sketch_1.addLine(1359.878972062445,      88,
1219.383030057759, 52.97042737240034)
Sketch_1.setCoincident(SketchLine_6.endPoint(),      SketchLine_7.startPoint(),
True)

### Create SketchLine
SketchLine_8 = Sketch_1.addLine(1219.383030057759, 52.97042737240034,
1219.383030057759, 0)
Sketch_1.setCoincident(SketchLine_7.endPoint(),      SketchLine_8.startPoint(),
True)
Sketch_1.setCoincident(SketchLine_4.endPoint(),      SketchLine_8.endPoint(),
True)
Sketch_1.setVertical(SketchLine_8.result(), True)

### Create SketchLine
SketchLine_9      =      Sketch_1.addLine(0,      196,      1183.003906157277,
50.74538483586779)

### Create SketchLine
SketchLine_10 = Sketch_1.addLine(1183.003906157277, 50.74538483586779,
1183.003906157277, 0)
Sketch_1.setCoincident(SketchLine_9.endPoint(),      SketchLine_10.startPoint(),
True)
Sketch_1.setCoincident(SketchLine_2.endPoint(),      SketchLine_10.endPoint(),
True)

### Create SketchLine
SketchLine_11      =      Sketch_1.addLine(1195.190840497792,      50,
1195.190840497792, 0)

```

```

Sketch_1.setCoincident(SketchLine_3.endPoint(), SketchLine_11.endPoint(),
True)
Sketch_1.setVertical(SketchLine_10.result(), True)
Sketch_1.setVertical(SketchLine_11.result(), True)

### Create SketchLine
SketchLine_12 = Sketch_1.addLine(1195.190840497792, 50,
1195.190840497792, 150)
SketchLine_12.setAuxiliary(True)
Sketch_1.setCoincident(SketchLine_11.startPoint(), SketchLine_12.startPoint(),
True)
Sketch_1.setVertical(SketchLine_12.result(), True)
Sketch_1.setVerticalDistance(SketchLine_12.endPoint(),
SketchLine_5.endPoint(), "OS", True)

### Create SketchConstraintAngle
Sketch_1.setAngle(SketchLine_9.result(), SketchLine_2.result(), "alpha/2", type
= "Direct", is_active = True)

### Create SketchConstraintAngle
Sketch_1.setAngle(SketchLine_7.result(), SketchLine_5.result(), "beta/2", type =
"Direct", is_active = True)
Sketch_1.setCoincident(SketchLine_9.startPoint(), SketchLine_1.startPoint(),
True)

### Create SketchArc
SketchArc_1 = Sketch_1.addArc(1195.190840497792, 150, 1183.003906157277,
50.74538483586779, 1195.190840497792, 50, False)
Sketch_1.setCoincident(SketchLine_12.endPoint(), SketchArc_1.center(), True)

```

```

Sketch_1.setCoincident(SketchLine_9.endPoint(), SketchArc_1.startPoint(),
True)
Sketch_1.setCoincident(SketchLine_11.startPoint(), SketchArc_1.endPoint(),
True)

### Create SketchArc
SketchArc_2 = Sketch_1.addArc(1195.190840497792, 150, 1219.383030057759,
52.97042737240034, 1195.190840497792, 50, True)
Sketch_1.setCoincident(SketchLine_12.endPoint(), SketchArc_2.center(), True)
Sketch_1.setCoincident(SketchLine_7.endPoint(), SketchArc_2.startPoint(),
True)
Sketch_1.setCoincident(SketchArc_2.endPoint(), SketchLine_11.startPoint(),
True)
Sketch_1.setTangent(SketchLine_7.result(), SketchArc_2.results()[1], True)
Sketch_1.setTangent(SketchArc_1.results()[1], SketchLine_9.result(), True)
Sketch_1.setVerticalDistance(SketchLine_11.startPoint(), SketchArc_1.center(),
"r", True)

### Create SketchProjection
SketchProjection_1 = Sketch_1.addProjection(model.selection("VERTEX",
"PartSet/Origin"), False)
SketchPoint_1 = SketchProjection_1.createdFeature()
Sketch_1.setCoincident(SketchLine_1.endPoint(),
SketchAPI_Point(SketchPoint_1).coordinates(), True)
Sketch_1.setVerticalDistance(SketchLine_7.startPoint(),
SketchLine_5.endPoint(), "d_outlet/2", True)
Sketch_1.setVerticalDistance(SketchLine_1.startPoint(),
SketchAPI_Point(SketchPoint_1).coordinates(), "d_inlet/2", True)

```

```

#### Create SketchLine
SketchLine_13      =      Sketch_1.addLine(1359.878972062445,      0,
3506.253778397836, 0)
Sketch_1.setCoincident(SketchLine_5.endPoint(),      SketchLine_13.startPoint(),
True)
Sketch_1.setHorizontal(SketchLine_13.result(), True)

#### Create SketchLine
SketchLine_14      =      Sketch_1.addLine(3506.253778397836,      0,
3506.253778397836, 88)
Sketch_1.setCoincident(SketchLine_13.endPoint(),      SketchLine_14.startPoint(),
True)
Sketch_1.setVertical(SketchLine_14.result(), True)

#### Create SketchLine
SketchLine_15      =      Sketch_1.addLine(3506.253778397836,      88,
1359.878972062445, 88)
Sketch_1.setCoincident(SketchLine_14.endPoint(),      SketchLine_15.startPoint(),
True)
Sketch_1.setCoincident(SketchLine_6.endPoint(),      SketchLine_15.endPoint(),
True)
Sketch_1.setHorizontal(SketchLine_15.result(), True)

#### Create SketchLine
SketchLine_16      =      Sketch_1.addLine(3506.253778397836,      88,
3506.253778397836, 792)
Sketch_1.setCoincident(SketchLine_14.endPoint(),      SketchLine_16.startPoint(),
True)
Sketch_1.setVertical(SketchLine_16.result(), True)

```

```

### Create SketchLine
SketchLine_17      =      Sketch_1.addLine(3506.253778397836,      792,
1359.878972059044, 792)
Sketch_1.setCoincident(SketchLine_16.endPoint(), SketchLine_17.startPoint(),
True)
Sketch_1.setHorizontal(SketchLine_17.result(), True)

### Create SketchLine
SketchLine_18      =      Sketch_1.addLine(1359.878972059044,      792,
1359.878972062445, 88)
Sketch_1.setCoincident(SketchLine_17.endPoint(), SketchLine_18.startPoint(),
True)
Sketch_1.setCoincident(SketchLine_6.endPoint(), SketchLine_18.endPoint(),
True)
Sketch_1.setVertical(SketchLine_18.result(), True)

### Create SketchLine
SketchLine_19      =      Sketch_1.addLine(1359.878972059044,      792,
1324.678972059044, 792)
Sketch_1.setCoincident(SketchLine_17.endPoint(), SketchLine_19.startPoint(),
True)
Sketch_1.setHorizontal(SketchLine_19.result(), True)

### Create SketchLine
SketchLine_20      =      Sketch_1.addLine(1324.678972059044,      792,
1324.678972062065, 88)
Sketch_1.setCoincident(SketchLine_19.endPoint(), SketchLine_20.startPoint(),
True)

```

```

Sketch_1.setVertical(SketchLine_20.result(), True)

### Create SketchLine
SketchLine_21      =      Sketch_1.addLine(1324.678972062065,      88,
1359.878972062445, 88)
Sketch_1.setCoincident(SketchLine_20.endPoint(), SketchLine_21.startPoint(),
True)
Sketch_1.setCoincident(SketchLine_6.endPoint(), SketchLine_21.endPoint(),
True)
Sketch_1.setHorizontal(SketchLine_21.result(), True)
Sketch_1.setHorizontalDistance(SketchLine_20.startPoint(),
SketchLine_18.startPoint(), "l2", True)
Sketch_1.setVerticalDistance(SketchLine_21.startPoint(),
SketchLine_20.startPoint(), "h", True)
Sketch_1.setLength(SketchLine_17.result(), "l1", True)
model.do()

### Create Face
Face_1_objects = [model.selection("FACE", "Sketch_1/Face-SketchLine_1r-
SketchLine_2f-SketchLine_10r-SketchLine_9r"),
model.selection("FACE", "Sketch_1/Face-SketchLine_3f-
SketchLine_11r-SketchArc_1_2r-SketchLine_10f"),
model.selection("FACE", "Sketch_1/Face-SketchLine_4f-
SketchLine_8r-SketchArc_2_2r-SketchLine_11f"),
model.selection("FACE", "Sketch_1/Face-SketchLine_5f-
SketchLine_6f-SketchLine_7f-SketchLine_8f"),
model.selection("FACE", "Sketch_1/Face-SketchLine_13f-
SketchLine_14f-SketchLine_15f-SketchLine_6r"),
model.selection("FACE", "Sketch_1/Face-SketchLine_16f-

```

```

SketchLine_17f-SketchLine_18f-SketchLine_15r"),
        model.selection("FACE",          "Sketch_1/Face-SketchLine_19f-
SketchLine_20f-SketchLine_21f-SketchLine_18r")]
Face_1 = model.addFace(Part_2_doc, Face_1_objects)

### Create Shell
Shell_1_objects = [model.selection("FACE", "Sketch_1/Face-SketchLine_1r-
SketchLine_2f-SketchLine_10r-SketchLine_9r"),
        model.selection("FACE",          "Sketch_1/Face-SketchLine_3f-
SketchLine_11r-SketchArc_1_2r-SketchLine_10f"),
        model.selection("FACE",          "Sketch_1/Face-SketchLine_4f-
SketchLine_8r-SketchArc_2_2r-SketchLine_11f"),
        model.selection("FACE",          "Sketch_1/Face-SketchLine_5f-
SketchLine_6f-SketchLine_7f-SketchLine_8f"),
        model.selection("FACE",          "Sketch_1/Face-SketchLine_13f-
SketchLine_14f-SketchLine_15f-SketchLine_6r"),
        model.selection("FACE",          "Sketch_1/Face-SketchLine_16f-
SketchLine_17f-SketchLine_18f-SketchLine_15r"),
        model.selection("FACE",          "Sketch_1/Face-SketchLine_19f-
SketchLine_20f-SketchLine_21f-SketchLine_18r")]
Shell_1 = model.addShell(Part_2_doc, Shell_1_objects)

### Create Revolution
Revolution_1 = model.addRevolution(Part_2_doc, [model.selection("SHELL",
"Shell_1_1")], model.selection("EDGE", "PartSet/OX"), 2.5, 2.5, "Faces")

### Create Group
Group_1_objects = [model.selection("EDGE",
"[Revolution_1_1_1/Generated_Face&Sketch_1/SketchLine_1][Revolution_1_1

```

```

_1/Generated_Face&Sketch_1/SketchLine_9])),
    model.selection("EDGE",
"[Revolution_1_1_1/Generated_Face&Sketch_1/SketchLine_10][Revolution_1_
1_1/Generated_Face&Sketch_1/SketchLine_9])),
    model.selection("EDGE",
"[Revolution_1_1_2/Generated_Face&Sketch_1/SketchLine_11][Revolution_1_
1_2/Generated_Face&Sketch_1/SketchArc_1_2])),
    model.selection("EDGE",
"[Revolution_1_1_3/Generated_Face&Sketch_1/SketchLine_8][Revolution_1_1
_3/Generated_Face&Sketch_1/SketchArc_2_2])),
    model.selection("EDGE",
"[Revolution_1_1_4/Generated_Face&Sketch_1/SketchLine_6][Revolution_1_1
_4/Generated_Face&Sketch_1/SketchLine_7])),
    model.selection("EDGE",
"[Revolution_1_1_7/Generated_Face&Sketch_1/SketchLine_20][Revolution_1_
1_7/Generated_Face&Sketch_1/SketchLine_21])),
    model.selection("EDGE",
"[Revolution_1_1_7/Generated_Face&Sketch_1/SketchLine_19][Revolution_1_
1_7/Generated_Face&Sketch_1/SketchLine_20])),
    model.selection("EDGE",
"[Revolution_1_1_7/Generated_Face&Sketch_1/SketchLine_19][Revolution_1_
1_7/Generated_Face&Sketch_1/SketchLine_18])),
    model.selection("EDGE",
"[Revolution_1_1_6/Generated_Face&Sketch_1/SketchLine_16][Revolution_1_
1_6/Generated_Face&Sketch_1/SketchLine_17])),
    model.selection("EDGE",
"[Revolution_1_1_6/Generated_Face&Sketch_1/SketchLine_16][Revolution_1_
1_6/Generated_Face&Sketch_1/SketchLine_15]"))
Group_1 = model.addGroup(Part_2_doc, "Edges", Group_1_objects)

```

Create Group

```
Group_2_objects = [model.selection("FACE",  
"Revolution_1_1_1/Generated_Face&Sketch_1/SketchLine_1"),  
model.selection("FACE",  
"Revolution_1_1_2/Generated_Face&Sketch_1/SketchLine_10"),  
model.selection("FACE",  
"Revolution_1_1_3/Generated_Face&Sketch_1/SketchLine_11"),  
model.selection("FACE",  
"Revolution_1_1_4/Generated_Face&Sketch_1/SketchLine_8"),  
model.selection("FACE",  
"Revolution_1_1_5/Generated_Face&Sketch_1/SketchLine_6"),  
model.selection("FACE",  
"Revolution_1_1_5/Generated_Face&Sketch_1/SketchLine_14")]  
Group_2 = model.addGroup(Part_2_doc, "Faces", Group_2_objects)
```

Create Group

```
Group_3_objects = [model.selection("EDGE",  
"[Revolution_1_1_1/Generated_Face&Sketch_1/SketchLine_9][Revolution_1_1_1/From_Face]"),  
model.selection("EDGE",  
"[Revolution_1_1_1/Generated_Face&Sketch_1/SketchLine_9][Revolution_1_1_1/To_Face]"),  
model.selection("EDGE",  
"[Revolution_1_1_1/From_Face][Revolution_1_1_1/To_Face]")]  
Group_3 = model.addGroup(Part_2_doc, "Edges", Group_3_objects)
```

Create Group

```
Group_4_objects = [model.selection("EDGE",
```

```

"[Revolution_1_1_2/Generated_Face&Sketch_1/SketchArc_1_2][Revolution_1_1_2/From_Face]"),
    model.selection("EDGE",
"[Revolution_1_1_2/Generated_Face&Sketch_1/SketchArc_1_2][Revolution_1_1_2/To_Face]"),
    model.selection("EDGE",
"[Revolution_1_1_2/From_Face][Revolution_1_1_2/To_Face]"))
Group_4 = model.addGroup(Part_2_doc, "Edges", Group_4_objects)

### Create Group
Group_5_objects = [model.selection("EDGE",
"[Revolution_1_1_3/Generated_Face&Sketch_1/SketchArc_2_2][Revolution_1_1_3/From_Face]"),
    model.selection("EDGE",
"[Revolution_1_1_3/Generated_Face&Sketch_1/SketchArc_2_2][Revolution_1_1_3/To_Face]"),
    model.selection("EDGE",
"[Revolution_1_1_3/From_Face][Revolution_1_1_3/To_Face]"))
Group_5 = model.addGroup(Part_2_doc, "Edges", Group_5_objects)

### Create Group
Group_6_objects = [model.selection("EDGE",
"[Revolution_1_1_4/Generated_Face&Sketch_1/SketchLine_7][Revolution_1_1_4/From_Face]"),
    model.selection("EDGE",
"[Revolution_1_1_4/Generated_Face&Sketch_1/SketchLine_7][Revolution_1_1_4/To_Face]"),
    model.selection("EDGE",
"[Revolution_1_1_4/From_Face][Revolution_1_1_4/To_Face]"))

```

```

Group_6 = model.addGroup(Part_2_doc, "Edges", Group_6_objects)

### Create Group
Group_7_objects = [model.selection("EDGE",
"[Revolution_1_1_6/Generated_Face&Sketch_1/SketchLine_17][Revolution_1_1_6/From_Face]"),
model.selection("EDGE",
"[Revolution_1_1_5/Generated_Face&Sketch_1/SketchLine_15][Revolution_1_1_5/To_Face]"),
model.selection("EDGE",
"[Revolution_1_1_5/From_Face][Revolution_1_1_5/To_Face]"),
model.selection("EDGE",
"[Revolution_1_1_6/Generated_Face&Sketch_1/SketchLine_17][Revolution_1_1_6/To_Face]"),
model.selection("EDGE",
"[Revolution_1_1_5/Generated_Face&Sketch_1/SketchLine_15][Revolution_1_1_5/From_Face]")]
Group_7 = model.addGroup(Part_2_doc, "Edges", Group_7_objects)

### Create Group
Group_8_objects = [model.selection("EDGE",
"[Revolution_1_1_7/Generated_Face&Sketch_1/SketchLine_19][Revolution_1_1_7/From_Face]"),
model.selection("EDGE",
"[Revolution_1_1_7/Generated_Face&Sketch_1/SketchLine_19][Revolution_1_1_7/To_Face]"),
model.selection("EDGE",
"[Revolution_1_1_7/Generated_Face&Sketch_1/SketchLine_21][Revolution_1_1_7/To_Face]"),

```

```

        model.selection("EDGE",
"[Revolution_1_1_7/Generated_Face&Sketch_1/SketchLine_21][Revolution_1_
1_7/From_Face]")])
Group_8 = model.addGroup(Part_2_doc, "Edges", Group_8_objects)

### Create Group
Group_9_objects = [model.selection("EDGE",
"[Revolution_1_1_6/Generated_Face&Sketch_1/SketchLine_16][Revolution_1_
1_6/From_Face]"),
        model.selection("EDGE",
"[Revolution_1_1_6/Generated_Face&Sketch_1/SketchLine_16][Revolution_1_
1_6/To_Face]"),
        model.selection("EDGE",
"[Revolution_1_1_7/Generated_Face&Sketch_1/SketchLine_18][Revolution_1_
1_7/To_Face]"),
        model.selection("EDGE",
"[Revolution_1_1_7/Generated_Face&Sketch_1/SketchLine_20][Revolution_1_
1_7/To_Face]"),
        model.selection("EDGE",
"[Revolution_1_1_7/Generated_Face&Sketch_1/SketchLine_20][Revolution_1_
1_7/From_Face]"),
        model.selection("EDGE",
"[Revolution_1_1_7/Generated_Face&Sketch_1/SketchLine_18][Revolution_1_
1_7/From_Face]")]
Group_9 = model.addGroup(Part_2_doc, "Edges", Group_9_objects)

### Create Group
Group_10_objects = [model.selection("FACE", "Revolution_1_1_1/To_Face"),
        model.selection("FACE", "Revolution_1_1_2/To_Face"),

```

```

        model.selection("FACE", "Revolution_1_1_3/To_Face"),
        model.selection("FACE", "Revolution_1_1_4/To_Face"),
        model.selection("FACE", "Revolution_1_1_5/To_Face"),
        model.selection("FACE", "Revolution_1_1_6/To_Face"),
        model.selection("FACE", "Revolution_1_1_7/To_Face")]
Group_10 = model.addGroup(Part_2_doc, "Faces", Group_10_objects)
Group_10.setName("front")
Group_10.result().setName("front")

### Create Group
Group_11_objects = [model.selection("FACE",
    "Revolution_1_1_1/Generated_Face&Sketch_1/SketchLine_9"),
    model.selection("FACE",
    "Revolution_1_1_2/Generated_Face&Sketch_1/SketchArc_1_2"),
    model.selection("FACE",
    "Revolution_1_1_3/Generated_Face&Sketch_1/SketchArc_2_2"),
    model.selection("FACE",
    "Revolution_1_1_4/Generated_Face&Sketch_1/SketchLine_7"),
    model.selection("FACE",
    "Revolution_1_1_7/Generated_Face&Sketch_1/SketchLine_21")]
Group_11 = model.addGroup(Part_2_doc, "Faces", Group_11_objects)
Group_11.setName("wall")
Group_11.result().setName("wall")

### Create Group
Group_12 = model.addGroup(Part_2_doc, "Faces", [model.selection("FACE",
    "Revolution_1_1_7/Generated_Face&Sketch_1/SketchLine_19"),
    model.selection("FACE",
    "Revolution_1_1_6/Generated_Face&Sketch_1/SketchLine_17")])

```

```

Group_12.setName("outletTop")
Group_12.result().setName("outletTop")

### Create Group
Group_13 = model.addGroup(Part_2_doc, "Faces", [model.selection("FACE",
"Revolution_1_1_1/Generated_Face&Sketch_1/SketchLine_1")])
Group_13.setName("inlet")
Group_13.result().setName("inlet")

### Create Group
Group_14 = model.addGroup(Part_2_doc, "Faces", [model.selection("FACE",
"Revolution_1_1_7/Generated_Face&Sketch_1/SketchLine_20")])
Group_14.setName("outletLeft")
Group_14.result().setName("outletLeft")

### Create Group
Group_15 = model.addGroup(Part_2_doc, "Faces", [model.selection("FACE",
"Revolution_1_1_6/Generated_Face&Sketch_1/SketchLine_16"),
model.selection("FACE",
"Revolution_1_1_5/Generated_Face&Sketch_1/SketchLine_14")])
Group_15.setName("outletRight")
Group_15.result().setName("outletRight")

### Create Group
Group_16_objects = [model.selection("FACE",
"Revolution_1_1_6/From_Face"),
                    model.selection("FACE", "Revolution_1_1_7/From_Face"),
                    model.selection("FACE", "Revolution_1_1_5/From_Face"),
                    model.selection("FACE", "Revolution_1_1_4/From_Face"),

```

```

        model.selection("FACE", "Revolution_1_1_3/From_Face"),
        model.selection("FACE", "Revolution_1_1_2/From_Face"),
        model.selection("FACE", "Revolution_1_1_1/From_Face")]
Group_16 = model.addGroup(Part_2_doc, "Faces", Group_16_objects)
Group_16.setName("back")
Group_16.result().setName("back")

model.end()

###
### SHAPERSTUDY component
###

model.publishToShaperStudy()
import SHAPERSTUDY
Face_1_1, = SHAPERSTUDY.shape(model.featureStringId(Face_1))
Face_1_2, = SHAPERSTUDY.shape(model.featureStringId(Face_1, 1))
Face_1_3, = SHAPERSTUDY.shape(model.featureStringId(Face_1, 2))
Face_1_4, = SHAPERSTUDY.shape(model.featureStringId(Face_1, 3))
Face_1_5, = SHAPERSTUDY.shape(model.featureStringId(Face_1, 4))
Face_1_6, = SHAPERSTUDY.shape(model.featureStringId(Face_1, 5))
Face_1_7, = SHAPERSTUDY.shape(model.featureStringId(Face_1, 6))
Revolution_1_1, Group_1_1, Group_2_1, Group_3_1, Group_4_1, Group_5_1,
Group_6_1, Group_7_1, Group_8_1, Group_9_1, front, wall, outletTop, inlet,
outletLeft,          outletRight,          back,          =
SHAPERSTUDY.shape(model.featureStringId(Revolution_1))

###
### SMESH component
###

```

```

import SMESH, SALOMEDS
from salome.smesh import smeshBuilder

smesh = smeshBuilder.New()

#smesh.SetEnablePublish( False ) # Set to False to avoid publish in study if not
needed or in some particular situations:

        # multiples meshes built in parallel, complex and numerous
mesh edition (performance)

Mesh_1 = smesh.Mesh(Revolution_1_1,'Mesh_1')
Regular_1D = Mesh_1.Segment()
Number_of_Segments_15 = Regular_1D.NumberOfSegments(NbSg1,None,[])
Prism_3D = Mesh_1.Prism()
front_1 = Mesh_1.GroupOnGeom(front,'front',SMESH.FACE)
wall_1 = Mesh_1.GroupOnGeom(wall,'wall',SMESH.FACE)
outletTop_1 = Mesh_1.GroupOnGeom(outletTop,'outletTop',SMESH.FACE)
inlet_1 = Mesh_1.GroupOnGeom(inlet,'inlet',SMESH.FACE)
outletLeft_1 = Mesh_1.GroupOnGeom(outletLeft,'outletLeft',SMESH.FACE)
outletRight_1 =
Mesh_1.GroupOnGeom(outletRight,'outletRight',SMESH.FACE)
Regular_1D_3 = smesh.CreateHypothesis('Regular_1D')
Prism_3D_4 = smesh.CreateHypothesis('Prism_3D')
Regular_1D_1 = Mesh_1.Segment(geom=Group_1_1)
Sub_mesh_1 = Regular_1D_1.GetSubMesh()
Number_of_Segments_1 = Regular_1D_1.NumberOfSegments(NbSg2,None,[])
Regular_1D_2 = Mesh_1.Segment(geom=Group_2_1)
status = Mesh_1.AddHypothesis(Number_of_Segments_15,Group_2_1)
Sub_mesh_2 = Regular_1D_2.GetSubMesh()

```

```

RadialQuadrangle_1D2D =
Mesh_1.Quadrangle(algo=smeshBuilder.RADIAL_QUAD,geom=Group_2_1)
isDone = Mesh_1.SetMeshOrder( [ [ Sub_mesh_1, Sub_mesh_2 ] ])
Regular_1D_3_1 = Mesh_1.Segment(geom=Group_3_1)
Number_of_Segments_100 =
Regular_1D_3_1.NumberOfSegments(NbSg3,None,[])
Number_of_Segments_100.SetConversionMode( 0 )
Number_of_Segments_100.SetExpressionFunction( '0.5*t' )
Regular_1D_4 = Mesh_1.Segment(geom=Group_4_1)
Number_of_Segments_2 = Regular_1D_4.NumberOfSegments(NbSg4,None,[])
Regular_1D_5 = Mesh_1.Segment(geom=Group_5_1)
Number_of_Segments_4 = Regular_1D_5.NumberOfSegments(NbSg5,None,[])
Regular_1D_6 = Mesh_1.Segment(geom=Group_6_1)
Number_of_Segments_30 = Regular_1D_6.NumberOfSegments(NbSg6,None,[])
Regular_1D_7 = Mesh_1.Segment(geom=Group_7_1)
Number_of_Segments_500 =
Regular_1D_7.NumberOfSegments(NbSg7,None,[])
Regular_1D_8 = Mesh_1.Segment(geom=Group_8_1)
Number_of_Segments_7 = Regular_1D_8.NumberOfSegments(NbSg8,None,[])
Regular_1D_9 = Mesh_1.Segment(geom=Group_9_1)
Number_of_Segments_56 = Regular_1D_9.NumberOfSegments(NbSg9,None,[])
Number_of_Segments_56.SetConversionMode( 0 )
Number_of_Segments_56.SetExpressionFunction( '-t*t' )

[ front_1, wall_1, outletTop_1, inlet_1, outletLeft_1, outletRight_1 ] =
Mesh_1.GetGroups()
isDone = Mesh_1.Compute()
Mesh_1.CheckCompute()
[ front_1, wall_1, outletTop_1, inlet_1, outletLeft_1, outletRight_1 ] =

```

```

Mesh_1.GetGroups()

back_1 = Mesh_1.GroupOnGeom(back,'back',SMESH.FACE)

[ front_1, wall_1, outletTop_1, inlet_1, outletLeft_1, outletRight_1, back_1 ] =
Mesh_1.GetGroups()

Sub_mesh_3 = Regular_1D_3_1.GetSubMesh()
Sub_mesh_4 = Regular_1D_4.GetSubMesh()
Sub_mesh_5 = Regular_1D_5.GetSubMesh()
Sub_mesh_6 = Regular_1D_6.GetSubMesh()
Sub_mesh_7 = Regular_1D_7.GetSubMesh()
Sub_mesh_8 = Regular_1D_8.GetSubMesh()
Sub_mesh_9 = Regular_1D_9.GetSubMesh()


try:
    # Mesh_1.ExportUNV( r'/home/matvey/OpenFOAM/matvey-
13/run/nozzle_1/geometry/Mesh.unv', 1 )
    Mesh_1.ExportUNV("/home/matvey/OpenFOAM/matvey-
13/run/shockFluidTube/geometry/Mesh.unv")
    pass
except Exception as e:
    print(e)
# except:
# print('ExportUNV() failed. Invalid file name?')


## Set names of Mesh objects
smesh.SetName(Number_of_Segments_2, 'Number of Segments_2')
smesh.SetName(front_1, 'front')
smesh.SetName(Sub_mesh_1, 'Sub-mesh_1')
smesh.SetName(Prism_3D_4, 'Prism_3D_4')

```

```
smesh.SetName(Sub_mesh_7, 'Sub-mesh_7')
smesh.SetName(Sub_mesh_5, 'Sub-mesh_5')
smesh.SetName(Sub_mesh_3, 'Sub-mesh_3')
smesh.SetName(Number_of_Segments_56, 'Number of Segments_56')
smesh.SetName(Sub_mesh_4, 'Sub-mesh_4')
smesh.SetName(Number_of_Segments_100, 'Number of Segments_100')
smesh.SetName(Mesh_1.GetMesh(), 'Mesh_1')
smesh.SetName(Number_of_Segments_15, 'Number of Segments_15')
smesh.SetName(Number_of_Segments_7, 'Number of Segments_7')
smesh.SetName(inlet_1, 'inlet')
smesh.SetName(outletRight_1, 'outletRight')
smesh.SetName(Sub_mesh_6, 'Sub-mesh_6')
smesh.SetName(outletLeft_1, 'outletLeft')
smesh.SetName(back_1, 'back')
smesh.SetName(RadialQuadrangle_1D2D.GetAlgorithm(),
'RadialQuadrangle_1D2D')
smesh.SetName(Number_of_Segments_30, 'Number of Segments_30')
smesh.SetName(Sub_mesh_9, 'Sub-mesh_9')
smesh.SetName(wall_1, 'wall')
smesh.SetName(Regular_1D_3, 'Regular_1D_3')
smesh.SetName(Number_of_Segments_4, 'Number of Segments_4')
smesh.SetName(Regular_1D.GetAlgorithm(), 'Regular_1D')
smesh.SetName(outletTop_1, 'outletTop')
smesh.SetName(Number_of_Segments_1, 'Number of Segments_1')
smesh.SetName(Number_of_Segments_500, 'Number of Segments_500')
smesh.SetName(Sub_mesh_2, 'Sub-mesh_2')
smesh.SetName(Prism_3D.GetAlgorithm(), 'Prism_3D')
smesh.SetName(Sub_mesh_8, 'Sub-mesh_8')
```

```
if salome.sg.hasDesktop():  
    salome.sg.updateObjBrowser()
```